

7.6.2 产品线的建立方式

软件产品线的建立需要企业有意识的、长期的努力才有可能成功。采用产品线方式开发软件时，开发人员可以划分为两组，分别是负责核心资源的小组和负责产品的小组。这也是产品线开发与独立系统开发的主要区别。

软件产品线的建立通常有4种方式，其划分依据有两个：第一个是企业采用演化方式（evolutionary）还是革命方式（revolutionary）引入产品线开发过程，第二个是企业基于现有产品开发还是开发全新的产品线。这4种方式的基本特征如表7-5所示。

表 7-5 软件产品线不同建立方式的基本特征

依据	演化方式	革命方式
基于现有产品	基于现有产品架构设计产品线的架构，经演化现有构件，开发产品线构件	核心资源的开发基于现有产品集的需求和可预测的、将来需求的超集
全新产品线	产品线核心资源随产品新成员的需求而演化	开发满足所有预期产品线成员的需求的核心资源

1. 将现有产品演化为产品线

在基于现有产品架构设计的产品线架构的基础上，将特定产品的构件逐步地、越来越多地转化为产品线的共用构件，从基于产品的方法“慢慢地”转化为基于产品线的软件开发。这种方法的主要优点是通过对投资回收期的分解、对现有系统演化的维持，使产品线方法的实施风险降到了最小，但完成产品线核心资源的总周期和总投资都比使用革命方式要大。

2. 用软件产品线替代现有产品集

基本停止现有产品的开发，所有工作直接针对软件产品线的核心资源开发。遗留系统只有在符合架构和构件需求的情况下，才可以和新的构件协作。这种方法的目标是开发一个不受现有产品集限制的、全新的平台，总周期和总投资较演化方式要少，但因重要需求的变化导致的初始投资报废的风险加大。另外，基于核心资源的第一个产品面世的时间将会推后。

除了投资和周期方面的考虑外，现有产品集中软硬件结合的紧密程度，以及不同产品在硬件方面的需求的差异，也是产品线开发采用演化还是革命方式的决策依据。对于软硬件结合密切，且硬件需求差异大的现有产品集，因无法满足产品线方法对软硬件同步的需求，只能采用革命方式替代现有产品集。

3. 全新软件产品线的演化

当企业进入一个全新的领域，要开发该领域的一系列产品时，同样也有演化和革命两种方式。演化方式将每一个新产品的需求与产品线核心资源进行协调。这种方式的好处是先期投资少，风险较小，第一个产品面世时间早。另外，因为是进入一个全新的领域，演化方式可以减少和简化因经验不足造成的初始阶段错误的修正代价。缺点是已有的产品线核心资源会影响新产品的需求协调，使成本加大。

4. 全新软件产品线的开发

设计师和开发工程师首先要得到产品线所有可能的需求，基于这个需求超集来设计和开发产品线核心资源。第一个产品将在产品线核心资源全部完成之后才开始构建。这种方式的优点是，一旦产品线核心资源完成后，新产品的开发速度将非常快，总成本也将减少；缺点是对新领域的需求很难做到全面和正确，使得核心资源不能像预期的那样支持新产品的开发。

从整体上来看，软件产品线的发展过程有三个阶段，分别是开发阶段、配置分发阶段和演化阶段。引起产品线架构演化的原因与引起任何其他系统演化的原因一样：产品线与技术变化的协调、现有问题的改正、新功能的增加、对现有功能的重组以允许更多的变化等。产品线的演化包括产品线核心资源的演化、产品的演化和产品的版本升级。

7.7 统一建模语言

统一建模语言（Unified Modeling Language, UML）是一种定义良好、易于表达、功能强大且普遍适用的建模语言，它融入了软件工程领域的新思想、新方法和新技术，它的作用域不限于支持面向对象分析和面向对象设计，还支持从需求分析开始的软件开发的全过程。

1. UML 的结构

从总体上来看，UML 的结构包括构造块、公共机制和规则三个部分。

（1）构造块。UML 有三种基本的构造块，分别是事物（thing）、关系（relationship）和图（diagram）。事物是 UML 的重要组成部分，关系把事物紧密联系在一起，图是多个相互关联的事物的集合。

（2）公共机制。公共机制是指达到特定目标的公共 UML 方法，主要包括规格说明（详细说明）、修饰、公共分类（通用划分）和扩展机制 4 种。规格说明是事物语义的细节描述，它是模型真正的核心；UML 为每个事物设置了一个简单的记号，还可以通过修饰来表达更多的信息；UML 包括两组公共分类，分别是类与对象（类表示概念，而对象表示具体的实体）、接口与实现（接口用来定义契约，而实现就是具体的内容）；扩展机制包括约束（扩展了 UML 构造块的语义，允许增加新的规则或修改现有的规则）、构造型（扩展 UML 的词汇，用于定义新的构造块）和标记值（扩展了 UML 构造块的特性，允许创建新的特殊信息来扩展事物的规格说明）。

（3）规则。规则是构造块如何放在一起的规定，具体包括：为构造块命名；赋予这些名称特定含义的上下文，即范围；怎样使用或看见名字，即可见性；事物如何正确、一致地相互联系，即完整性；运行或模拟动态模型的含义是什么，即执行。

UML 对系统架构的定义是系统的组织结构，包括系统分解的组成部分，以及它们的关联性、交互机制和指导原则等提供系统设计的信息。具体来说，就是指以下 5 个系统视图：

（1）逻辑视图。逻辑视图也称为设计视图，它表示了设计模型中在架构方面具有重要意义的部分，即类、子系统、包和用例实现的子集。

（2）进程视图。进程视图是可执行线程和进程作为活动类的建模，它是逻辑视图的一次执

行实例，描述了并发与同步结构。

(3) 实现视图。实现视图对组成基于系统的物理代码的文件和构件进行建模。

(4) 部署视图。部署视图把构件部署到一组物理节点上，表示软件到硬件的映射和分布结构。

(5) 用例视图。用例视图是最基本的需求分析模型。

另外，UML 还允许在一定的阶段隐藏模型的某些元素、遗漏某些元素，以及不保证模型的完整性，但模型要逐步地达到完整和一致。

2. 事物

UML 中的事物也称为建模元素，包括结构事物（Structural Things）、行为事物（Behavioral Things，动作事物）、分组事物（Grouping Things）和注释事物（Annotational Things，注解事物）。这些事物是 UML 模型中最基本的 OO 构造块。

(1) 结构事物。结构事物在模型中属于最静态的部分，代表概念上或物理上的元素。UML 有 7 种结构事物，分别是类、接口、协作、用例、活动类、构件和节点。类是描述具有相同属性、方法、关系和语义的对象的集合，一个类实现一个或多个接口；接口是指类或构件提供特定服务的一组操作的集合，接口描述了类或构件的对外的可见的动作；协作定义了交互的操作，是一些角色和其他事物一起工作，提供一些合作的动作，这些动作比事物的总和要大；用例是描述一系列的动作，产生有价值的结果，在模型中用例通常用来组织行为事物，用例是通过协作来实现的；活动类的对象有一个或多个进程或线程，活动类和类很相似，只是它的对象代表的事物的行为和其他事物是同时存在的；构件是物理上或可替换的系统部分，它实现了一个接口集合；节点是一个物理元素，它在运行时存在，代表一个可计算的资源，通常占用一些内存且具有处理能力，一个构件集合一般来说位于一个节点，但有可能从一个节点转到另一个节点。

(2) 行为事物。行为事物是 UML 模型中的动态部分，代表时间和空间上的动作。UML 有两种主要的行为事物。第一种是交互（内部活动），交互是由一组对象之间在特定上下文中，为达到特定目的而进行的一系列消息交换而组成的动作。交互中组成动作的对象的每个操作都要详细列出，包括消息、动作次序（消息产生的动作）、连接（对象之间的连接）。第二种是状态机，状态机由一系列对象的状态组成。

(3) 分组事物。分组事物是 UML 模型中组织的部分，可以把它们看成是个盒子，模型可以在其中进行分解。UML 只有一种分组事物，称为包。包是一种将有组织的元素分组的机制。与构件不同的是，包纯粹是一种概念上的事物，只存在于开发阶段，而构件可以存在于系统运行阶段。

(4) 注释事物。注释事物是 UML 模型的解释部分。

3. 关系

UML 用关系把事物结合在一起，主要有下列 4 种关系：

(1) 依赖（dependency）。依赖是两个事物之间的语义关系，其中一个事物发生变化会影响另一个事物的语义。

(2) 关联（association）。关联描述一组对象之间连接的结构关系。

(3) 泛化 (generalization)。泛化是一般化和特殊化的关系，描述特殊元素的对象可替换一般元素的对象。

(4) 实现 (realization)。实现是类之间的语义关系，其中的一个类指定了由另一个类保证执行的契约。

4. 图

UML 2.0 包括 14 种图，分别列举如下：

(1) 类图 (Class Diagram)。类图描述一组类、接口、协作和它们之间的关系。在 OO 系统的建模中，最常见的图就是类图。类图给出了系统的静态设计视图，活动类的类图给出了系统的静态进程视图。

(2) 对象图 (Object Diagram)。对象图描述一组对象及它们之间的关系。对象图描述了在类图中所建立的事物实例的静态快照。和类图一样，这些图给出系统的静态设计视图或静态进程视图，但它们是从真实案例或原型案例的角度建立的。

(3) 构件图 (Component Diagram)。构件图描述一个封装的类和它的接口、端口，以及由内嵌的构件和连接件构成的内部结构。构件图用于表示系统的静态设计实现视图。对于由小的部件构建大的系统来说，构件图是很重要的。构件图是类图的变体。

(4) 组合结构图 (Composite Structure Diagram)。组合结构图描述结构化类（例如，构件或类）的内部结构，包括结构化类与系统其余部分的交互点。组合结构图用于画出结构化类的内部内容。

(5) 用例图 (Use Case Diagram)。用例图描述一组用例、参与者及它们之间的关系。用例图给出系统的静态用例视图。这些图在对系统的行为进行组织和建模时是非常重要的。

(6) 顺序图 (Sequence Diagram，也称为序列图)。顺序图是一种交互图 (Interaction Diagram)，交互图展现了一种交互，它由一组对象或参与者以及它们之间可能发送的消息构成。交互图专注于系统的动态视图。顺序图是强调消息的时间次序的交互图。

(7) 通信图 (Communication Diagram)。通信图也是一种交互图，它强调收发消息的对象或参与者的组织结构。顺序图和通信图表达了类似的基本概念，但它们所强调的概念不同，顺序图强调的是时序，通信图强调的是对象之间的组织结构（关系）。在 UML 1.X 版本中，通信图称为协作图 (Collaboration Diagram)。

(8) 定时图 (Timing Diagram，计时图)。定时图也是一种交互图，它强调消息跨越不同对象或参与者的实际时间，而不仅仅关心消息的相对顺序。

(9) 状态图 (State Diagram)。状态图描述一个状态机，它由状态、转移、事件和活动组成。状态图给出了对象的动态视图。它对于接口、类或协作的行为建模尤为重要，而且它强调事件导致的对象行为，这非常有助于对反应式系统建模。

(10) 活动图 (Activity Diagram)。活动图将进程或其他计算结构展示为计算内部的控制流和数据流。活动图专注于系统的动态视图。它对系统的功能建模和业务流程建模特别重要，并强调对象间的控制流程。

(11) 部署图 (Deployment Diagram)。部署图描述对运行时的处理节点及在其中生存的构

件的配置。部署图给出了架构的静态部署视图，通常一个节点包含一个或多个部署图。

(12) 制品图 (Artifact Diagram)。制品图描述计算机中一个系统的物理结构。制品包括文件、数据库和类似的物理比特集合。制品图通常与部署图一起使用。制品也给出了它们实现的类和构件。

(13) 包图 (Package Diagram)。包图描述由模型本身分解而成的组织单元，以及它们之间的依赖关系。

(14) 交互概览图 (Interaction Overview Diagram)。交互概览图是活动图和顺序图的混合物。

7.8 软件形式化方法

提高软件可靠性的一种重要技术是使用形式化方法。形式化方法是建立在严格数学基础上、具有精确数学语义的开发方法。广义的形式化方法是指软件开发过程中分析、设计和实现的系统工程方法，狭义的形式化方法是指软件规格和验证的方法。

1. 形式化方法概述

近年来，形式化方法已不再只是一种研究所里的学术研究工作，而是已经开始被工业界接受并应用于开发实际的系统。例如，在需求分析中，形式化方法的思想是利用形式化规格说明语言，严格定义用户需求，并采用数学推演的方法证明需求定义的性质，对于复杂的应用问题，尽管无法验证整个需求定义的完整性，但仍有可能为避免某些要点的疏漏而建立数学断言，然后予以形式证明或反驳。形式化规格说明语言包括严格的语法定义和语义定义，以及一系列的数学推演规则。这些规则不仅说明了某些数学性质在软件规格说明中是否成立，也说明了软件实现与软件规格说明之间的满足关系。

形式化方法的主要优越性在于它能够数学地表述和研究应用问题及其软件实现。但是，它要求开发人员具备良好的数学基础。用形式化语言书写的大型应用问题的软件规格说明往往过于细节化，并且难于为用户和软件设计人员所理解。由于这些缺陷，形式化方法在目前的软件开发实践中并未得到普遍应用。但是，近年来，形式化方法在以下两个方面的发展大大改善了其实用性：

(1) 形式化方法与图形语言机制相结合。为图形语言机制赋予形式化的语法和语义，从而兼具了图形表示的直观、简洁，以及形式化方法的严谨、精确等优点。

(2) 用 CASE (Computer Aided Software Engineering, 计算机辅助软件工程) 工具支持形式化软件开发。CASE 工具不仅可以简化描述工作，而且可以利用自动证明技术，帮助开发人员验证软件的数学性质。

实践证明，上述技术途径对于克服形式化方法的主要缺陷是行之有效的，因此，它们将在形式化方法的未来发展中发挥重要作用。

2. 净室软件工程

净室软件工程 (Cleanroom Software Engineering, CSE) 是软件开发的一种形式化方法，可

以开发较高质量的软件。它使用盒结构归约进行分析和建模，并且将正确性验证作为发现和排除错误的主要机制，使用统计测试来获取认证软件可靠性所需要的信息。

CSE 强调在规约和设计上的严格性，以及使用基于数学的正确性证明来对设计模型的每个元素进行形式化验证。作为对形式化方法的扩展，CSE 还强调统计质量控制技术，包括基于客户对软件的预期使用的测试。

CSE 的理论基础是函数理论和抽样理论，所采用的技术手段主要有以下 4 个方面：

(1) 统计过程控制下的增量式开发。

(2) 基于函数的规范、设计。CSE 按照函数理论定义了三种抽象层次，分别是行为视图、有限状态机视图和过程视图。规范从一个外部行为视图（称为黑盒）开始，然后被转化为一个状态机视图（称为状态盒），最后由一个过程视图（明盒）来实现。盒结构是基于对象的，并支持软件工程的关键原则，即信息隐藏、接口与实现分离。

(3) 正确性验证。正确性验证是 CSE 的核心，正是由于采用了这一技术，软件质量才有了极大的提高。

(4) 统计测试和软件认证。CSE 在测试方面采用统计学的基本原理，即当总体太大时必须采取抽样的方法。首先，确定一个使用模型来代表系统所有可能使用的（一般是无限的）总体；然后，由使用模型产生测试用例，因为测试用例是总体的一个随机样本，所以可得到系统预期操作性能的有效的统计推导。

3. 程序验证

在软件工程领域，用于确保程序正确性的手段有很多，大体可分为两类：动态方法（Dynamic）和静态方法（Static）。动态方法需要将软件源码编译为可执行文件，再运行软件，通过特定的输入考察软件的运行结果。动态方法中使用得比较广泛的是软件测试，有手动测试、自动化测试等。动态方法效率高，错误定位准确，但应用场合受限，它要求必须具有程序能运行起来的环境。而且，其最大的问题在于，动态方法不能覆盖程序的所有输入和全部的执行路径，故只能证明在已有的输入下，被测试所覆盖的部分程序没有问题，而不能证明整个程序不存在错误。静态方法则是基于程序源代码进行分析，无须编译运行，所以应用场景更加广泛。常用的静态方法有静态分析、符号执行、定理证明、模型检测等。不同的静态方法各有优劣，其应用门槛也比较高，一般用于对程序正确性要求较高的场景。

(1) 静态分析（Static Analysis）方法比较轻量级，适用于大规模代码，但比较大的问题是较高比例的误报（False Positive）。所以静态分析报出的结果需要逐一人工排查，从而导致耗费大量的人力资源。比较有名的静态分析引擎如 Coverity，目前已经实现较大规模商业化应用。

(2) 符号执行（Symbolic Execution）可以看作是更加准确的测试方法，它通过符号值来静态执行程序，积累路径条件（Path Condition），直到到达目标位置，再对路径条件进行约束求解（Constraint Solving），判断目标位置的可达性（Reachability）。由于需要使用约束求解，而且对循环不友好，所以符号执行方法比较难以大规模应用。经典的符号执行工具如 KLEE，已经被加入 LLVM 的官方项目列表中。

(3) 定理证明（Theorem Proving）方法是使用高阶逻辑（High Order Logic, HOL）对程

序及其需要满足的性质进行建模描述，然后使用机器辅助证明的方法，一步一步证明程序能够满足要求的性质。定理证明方法的主要缺陷是自动化程度较低，需要大量的专业人力参与，编写证明代码，对软件的快速更新迭代不友好。辅助定理证明的典型工具是 Coq，于 2014 年获得 ACM 软件系统奖。此外，值得一提的是，基于定理证明方法验证的 C 语言编译工具链 CompCert，于 2021 年获得 ACM 软件系统奖。

（4）模型检测（Model Checking）是一种经典的形式化分析方法。它通过构造软件系统的抽象模型，来检测其是否满足要求的性质。模型检测方法的缺点是系统模型的建立需要领域专家的参与，寻找恰当的抽象层次，从而足以证明系统的特定属性是模型检测的一大难点。过分的抽象将导致属性无法证明，而不足的抽象又将导致太多属性无关的冗余细节，从而引发状态爆炸，无法在合理的时间内得到结果。经典的模型检测工具有 NuSMV、SPIN 等。其中，SPIN 于 2002 年获得 ACM 软件系统奖。此外，还有基于模型驱动开发（Model Driven Development）的 B 方法，以及其对应的工具链 Event B。

第8章 项目管理

在现代项目实践中，由于项目管理在项目实施过程中发挥重大作用，其已经成为每个项目必不可少的工作内容。项目管理是保证项目成功的核心手段，成熟的项目管理体系是项目经理的得力工具。因此，作为信息系统项目中的骨干成员，系统分析师必须熟悉项目管理知识，掌握项目管理的主要过程、方法、技术和工具。

8.1 项目管理概述

项目是在特定条件下，具有特定目标的一次性任务，是在一定时间内，满足一系列特定目标的多项相关工作的总称。从项目的基本定义出发，人们日常进行的所有活动都可定义或分解为项目。

项目管理是管理学的一个分支学科，是指在项目活动中运用专门的知识、技能、工具和方法，使项目能够在有限资源限定条件下，实现或超过设定的需求和期望的过程。项目管理是对一些成功达成一系列目标相关活动（譬如任务）的整体监测和管控，包括策划、进度计划和维护组成项目的活动的进展。

软件项目管理的对象是软件工程项目。它所涉及的范围覆盖了整个软件工程过程。为使软件项目开发获得成功，关键问题是必须对软件项目的工作范围、可能风险、需要资源（人、硬件/软件）、要实现的任务、经历的里程碑、花费工作量（成本）、进度安排等做到心中有数。这种管理在技术工作开始之前开始，在软件从概念到实现的过程中继续进行，当软件工程过程最后结束时才终止。

软件项目管理是为了使软件项目能够按照预定的成本、进度、质量顺利完成，而对人员（People）、产品（Product）、过程（Process）和项目（Project）等进行分析和管理的活动。

8.2 范围管理

范围是项目目标的更具体的表达。在信息系统项目实践中，需求无序扩张是项目失败最常见的原因之一，往往在项目启动、计划、执行，甚至收尾时还在不断地加入新的功能。无论是客户的要求，还是项目团队成员对新技术的试验，都可能导致项目范围的失控，从而使项目的时间、资源和质量上都受到严重影响。

范围管理就是要确定项目的边界，也就是说，要确定哪些工作是本次项目应该做的，哪些工作不应该包括在本次项目中。这个过程用于确保项目干系人对作为项目结果的产品（或服务），以及开发这些产品所确定的过程有一个共同的理解。